

IUT d'Aix-Marseille – Département Informatique

BUT Informatique – 2^e année

R3.07 – Bases de données

TD1 : rappels de SQL

Travail sur feuille

Base support : `mini-pagila` (version réduite de Pagila)

Version étudiants

7 septembre 2026

Déroulement de la séance

Cette séance est un **rappel** du SQL vu en première année. Elle se déroule **sur feuille**, sans SGBD : vous ne pourrez donc pas « tester pour voir ». C'est volontaire : l'objectif est précisément d'être capable de *prédire* le comportement d'une requête et de *justifier* sa correction, compétence indispensable dès que les requêtes deviennent complexes (et indispensable, aussi, pour relire du SQL écrit par quelqu'un d'autre... ou par une IA générative).

Toutes les questions portent sur la base **mini-pagila**, décrite en **annexe A** (page 9) : schéma et contenu complet des tables. *Gardez cette annexe sous les yeux pendant toute la séance*. Sauf mention contraire, on utilise la syntaxe **PostgreSQL**.

Le sujet est organisé en trois sections thématiques :

Section 1 – requêtes de base **SELECT-FROM-WHERE**, tris et premières sous-requêtes ;

Section 2 – jointures ;

Section 3 – groupements et agrégations.

Chaque section combine cinq types d'exercices : *lire* une requête, *juger* un résultat, *compléter* ou *corriger* une requête, *écrire* une requête, et *reconstruire* une requête à partir de son résultat. Les questions marquées d'une étoile (*) sont des approfondissements : traitez-les si vous avez terminé le reste de la section.

Aide-mémoire : ordre d'évaluation logique des clauses

$\underbrace{\text{FROM}}_1 \rightarrow \underbrace{\text{WHERE}}_2 \rightarrow \underbrace{\text{GROUP BY}}_3 \rightarrow \underbrace{\text{HAVING}}_4 \rightarrow \underbrace{\text{SELECT}}_5 \rightarrow \underbrace{\text{ORDER BY}}_6 \rightarrow \underbrace{\text{LIMIT}}_7$

Conséquences à ne jamais oublier :

- **WHERE** filtre *des lignes avant* regroupement ; **HAVING** filtre *des groupes après* regroupement. Un agrégat (**COUNT**, **SUM**, ...) ne peut donc pas apparaître dans **WHERE**.
- Les alias définis dans **SELECT** sont utilisables dans **ORDER BY** (évalué après), pas dans **WHERE**.
- **NULL** n'est égal à rien, pas même à **NULL** : toute comparaison avec **NULL** vaut *inconnu*, et une ligne dont la condition vaut *inconnu* n'est pas retenue. On teste avec **IS NULL** / **IS NOT NULL**.
- Les fonctions d'agrégation (sauf **COUNT(*)**) **ignorent** les **NULL**.

1 Requêtes de base : SELECT – FROM – WHERE

Exercice 1.1 – Lire une requête (SQL → français)

Pour chacune des requêtes suivantes : (i) rédigez en **une phrase** l'énoncé en français dont elle est la réponse – comme si vous l'expliquiez au client qui l'a commandée, sans paraphraser le SQL clause par clause; (ii) donnez son résultat sur l'instance de l'annexe A.

- a)

```
SELECT title, length
FROM film
WHERE rating = 'R'
ORDER BY length DESC;
```
- b)

```
SELECT first_name, last_name
FROM customer
WHERE city IN ('Marseille', 'Toulon')
AND active = TRUE;
```
- c)

```
SELECT title
FROM film
WHERE length IS NULL OR length < 80;
```

Exercice 1.2 – Compléter la requête

Complétez les trous numérotés pour que la requête réponde à l'énoncé. Un trou peut contenir plusieurs mots.

- a) *Titres et années des films sortis entre 2010 et 2015 (bornes incluses), du plus ancien au plus récent.*

```
SELECT title, release_year
FROM film
WHERE release_year .....(1)..... 2010 .....(2)..... 2015
ORDER BY .....(3).....;
```

- b) *Nom, prénom et date d'inscription des clients actifs d'Aix-en-Provence, du plus ancien inscrit au plus récent.*

```
SELECT first_name, last_name, .....(1).....
FROM customer
WHERE city = .....(2).....
.....(3)..... active = TRUE
ORDER BY create_date .....(4).....;
```

- c) *Titre et tarif du (ou des) film(s) le(s) plus cher(s) du catalogue.*

```
SELECT title, rental_rate
FROM film
WHERE rental_rate = ( SELECT .....(1)..... FROM .....(2)..... );
```

Exercice 1.3 – Corriger la requête

Chacune des requêtes ci-dessous est fautive : soit elle produit une erreur, soit elle ne répond pas à l'énoncé. Identifiez le problème, expliquez-le, puis réécrivez la requête correcte.

- a) *Titres des films dont la durée n'est pas renseignée.*

```
SELECT title FROM film WHERE length = NULL;
```

- b) *Titres des films dont le tarif dépasse le tarif moyen du catalogue.*

```
| SELECT title FROM film WHERE rental_rate > AVG(rental_rate);
```

c) *Titres des films d'horreur.*

```
| SELECT title  
| FROM film  
| WHERE category_id IN (SELECT * FROM category WHERE name = 'Horreur');
```

Exercice 1.4 – Écrire la requête

Écrivez les requêtes répondant aux énoncés suivants.

- a) Titres des films tous publics (classification G ou PG) durant au moins 90 minutes, par ordre alphabétique.
- b) Liste des villes où résident des clients, sans doublon, par ordre alphabétique.
- c) Titre et durée des trois films les plus longs du catalogue. *Attention aux durées inconnues.*

2 Jointures

Exercice 2.1 – Lire une requête (SQL → français)

Même consigne qu'en 1.1 : énoncé en français en une phrase, puis résultat sur l'instance de l'annexe A. Précisez à chaque fois le **nombre de lignes** du résultat.

- a)

```
SELECT f.title, c.name
FROM   film f
      JOIN category c ON f.category_id = c.category_id
WHERE  c.name = 'Documentaire'
ORDER BY f.title;
```
- b)

```
SELECT f.title
FROM   film f
      LEFT JOIN rental r ON f.film_id = r.film_id
WHERE  r.rental_id IS NULL;
```
- c)

```
SELECT a.first_name, a.last_name, f.title
FROM   actor a
      JOIN film_actor fa ON a.actor_id = fa.actor_id
      JOIN film f       ON fa.film_id = f.film_id
WHERE  f.release_year = 2010
ORDER BY a.last_name;
```

Exercice 2.2 – Compléter, puis écrire

Partie 1 – à compléter.

- a) *Prénom, nom du client, titre du film et date de location, pour toutes les locations en cours (non rendues), de la plus ancienne à la plus récente.*

```
SELECT cu.first_name, cu.last_name, f.title, r.rental_date
FROM   rental r
      JOIN customer cu ON .....(1).....
      JOIN film f      ON .....(2).....
WHERE  r.return_date .....(3).....
ORDER BY .....(4).....;
```

- b) *Titres des films de la catégorie « Comédie » qui ne sont pas actuellement loués.*

```
SELECT f.title
FROM   film f
      JOIN category ca ON f.category_id = ca.category_id
      .....(1)..... rental r ON .....(2).....
WHERE  ca.name = 'Comédie'
      AND ( .....(3)..... OR r.film_id IS NULL);
```

Partie 2 – à écrire entièrement.

- c) Prénom et nom des acteurs ayant joué dans au moins un film d'action, sans doublon, par ordre alphabétique de nom.
- d) Titres des documentaires loués par un client habitant Marseille.
- e) Noms des acteurs ayant partagé l'affiche d'au moins un film avec Nick Wahlberg (sans le citer lui-même).

Exercice 2.3 – Regard critique sur une requête générée par IA

Un camarade a demandé à un assistant IA d'écrire les requêtes correspondant aux énoncés suivants. Pour chaque cas : **(i)** déterminez si la requête répond correctement à l'énoncé ; **(ii)** donnez son résultat sur l'instance de l'annexe A ; **(iii)** si nécessaire, proposez une version correcte. Rédigez votre analyse comme une revue de code : on doit pouvoir vous croire sur parole, sans exécuter la requête.

Cas A – Énoncé : « Donner la liste des clients (prénom, nom) ayant loué au moins un film d'horreur. Un client ne doit apparaître qu'une fois. »

Requête proposée par une IA générative

```
SELECT cu.first_name, cu.last_name
FROM   customer cu
       JOIN rental r      ON cu.customer_id = r.customer_id
       JOIN film f        ON r.film_id      = f.film_id
       JOIN category ca   ON f.category_id = ca.category_id
WHERE  ca.name = 'Horreur';
```

Question complémentaire : on ajoute la location (19, '2025-04-02', NULL, 10, 2) (le client 2 loue Jungle Closer). Votre réponse au (i) change-t-elle ?

Cas B – Énoncé : « Donner la liste des clients (prénom, nom) n'ayant jamais loué de film d'horreur. »

Requête proposée par une IA générative

```
SELECT cu.first_name, cu.last_name
FROM   customer cu
       LEFT JOIN rental r      ON cu.customer_id = r.customer_id
       LEFT JOIN film f        ON r.film_id      = f.film_id
       LEFT JOIN category ca   ON f.category_id = ca.category_id
WHERE  ca.name = 'Horreur' AND r.rental_id IS NULL;
```

3 Groupements et agrégations

Exercice 3.1 – Lire une requête (SQL → français)

Même consigne qu'en 1.1 et 2.1. Pour chaque requête, précisez en outre **ce que représente une ligne** du résultat.

- a) `SELECT ca.name, COUNT(*) AS nb
FROM film f JOIN category ca ON f.category_id = ca.category_id
GROUP BY ca.name
HAVING COUNT(*) >= 3
ORDER BY nb DESC;`
- b) `SELECT rating, AVG(length) AS duree_moy
FROM film
WHERE release_year >= 2010
GROUP BY rating
ORDER BY duree_moy DESC;`
- c) `SELECT customer_id, COUNT(*) AS nb_locations, COUNT(return_date) AS nb_rendues
FROM rental
GROUP BY customer_id
ORDER BY customer_id;`
- d) `SELECT ca.name, AVG(f.rental_rate)
FROM film f JOIN category ca ON f.category_id = ca.category_id
GROUP BY ca.name
HAVING AVG(f.rental_rate) > (SELECT AVG(rental_rate) FROM film);`

Exercice 3.2 – Ce résultat est-il cohérent ?

Même consigne qu'en 1.2.

Cas A

```
SELECT cu.customer_id, cu.last_name, SUM(p.amount) AS total_paye
FROM customer cu JOIN payment p ON cu.customer_id = p.customer_id
GROUP BY cu.customer_id, cu.last_name
ORDER BY cu.customer_id;
```

Résultat proposé

customer_id	last_name	total_paye
1	Dupont	10.97
2	Martin	0.99
3	Bernard	4.99
4	Petit	8.98
5	Moreau	8.97
6	Girard	6.97
7	Roche	2.99
8	Faure	0.00

Cas B

```
SELECT rating, COUNT(*) AS nb, AVG(length) AS duree_moy
FROM film
GROUP BY rating
ORDER BY rating;
```

Résultat proposé

	rating	nb	duree_moy
G	3	102.00	
PG	4	102.25	
PG-13	2	51.50	
R	3	111.33	

Cas C

```
SELECT ca.name, COUNT(*) AS nb, ROUND(AVG(f.rental_rate), 2) AS tarif_moyen
FROM film f JOIN category ca ON f.category_id = ca.category_id
GROUP BY ca.name
HAVING COUNT(*) > 2
ORDER BY ca.name;
```

Résultat proposé

name	nb	tarif_moyen
Action	3	4.99
Documentaire	3	3.32

Exercice 3.3 – Corriger, puis écrire

Partie 1 – à corriger. Identifiez l'erreur, expliquez-la, réécrivez.

- a) *Identifiant des catégories comptant au moins 3 films, et leur nombre de films.*

```
SELECT category_id, COUNT(*)
FROM film
WHERE COUNT(*) >= 3
GROUP BY category_id;
```

- b) *Durée moyenne des films par classification.*

```
SELECT rating, AVG(length) FROM film;
```

Partie 2 – à écrire entièrement.

- c) Nom de chaque catégorie et nombre de films qu'elle contient, de la plus fournie à la moins fournie.
- d) Nom de chaque client et son nombre de locations, **y compris les clients n'ayant jamais rien loué** (qui doivent apparaître avec 0), du plus gros loueur au plus petit.
- e) Titres des films loués au moins 3 fois, avec leur nombre de locations.
- f) Nom des clients ayant payé plus de 5 € au total, avec le montant, par montant décroissant.

Exercice 3.4 – Reconstruire la requête à partir du résultat

Pour chacun des résultats ci-dessous, obtenu sur l'instance de l'annexe A : (i) formulez l'énoncé en français, (ii) écrivez une requête produisant *exactement* ce résultat, ordre des lignes compris.

Résultat 1

Résultat 1

name	nb_films	duree_moy
Action	3	113.0
Animation	2	91.0
Comédie	2	97.5
Documentaire	3	115.0
Horreur	2	103.0

Résultat 2

Résultat 2

title	nb_acteurs
Lonely Elephant	0
Chocolate Duck	1
Frost Head	1
Glacier Ocean	1
Island Empire	1
Kissing Dolls	1

Exercice 3.5 – Regard critique sur une requête générée par IA

Même consigne qu'en 2.3 : la requête répond-elle à l'énoncé ? Quel est son résultat ? Quelle version proposez-vous ?

Cas A – Énoncé : « Pour chaque client, donner son nom et le nombre de locations qu'il n'a pas encore rendues. Tous les clients doivent apparaître, y compris ceux qui n'ont aucune location en cours. »

Requête proposée par une IA générative

```
SELECT cu.last_name, COUNT(*) AS nb_non_rendues
FROM   customer cu JOIN rental r ON cu.customer_id = r.customer_id
WHERE  r.return_date IS NULL
GROUP BY cu.customer_id, cu.last_name;
```

Un camarade repère le problème et propose le correctif suivant ; qu'en pensez-vous ?

Correctif proposé

```
SELECT cu.last_name, COUNT(*) AS nb_non_rendues
FROM   customer cu LEFT JOIN rental r ON cu.customer_id = r.customer_id
WHERE  r.return_date IS NULL
GROUP BY cu.customer_id, cu.last_name;
```

Cas B – Énoncé : « Quelle est la catégorie contenant le plus de films ? »

Requête proposée par une IA générative

```
SELECT ca.name, COUNT(*) AS nb
FROM   film f JOIN category ca ON f.category_id = ca.category_id
GROUP BY ca.name
ORDER BY nb DESC
LIMIT 1;
```

A La base mini-pagila

mini-pagila est une version réduite de la base Pagila (location de films) que vous utiliserez en TP sous PostgreSQL. Les noms de tables et de colonnes sont ceux de Pagila ; le schéma a simplement été allégé pour tenir sur une feuille.

Schéma relationnel

```
category    (category_id, name)
film        (film_id, title, release_year, length, rating, rental_rate, category_id)
actor       (actor_id, first_name, last_name)
film_actor  (film_id, actor_id)
customer    (customer_id, first_name, last_name, city, active, create_date)
rental      (rental_id, rental_date, return_date, film_id, customer_id)
payment     (payment_id, customer_id, rental_id, amount, payment_date)
```

Clés primaires soulignées, clés étrangères en *italique*. Seules `film.length` et `rental.return_date` peuvent valoir NULL (`return_date` NULL = location en cours). `rental_rate` et `amount` sont en euros ; `active` est un booléen.

Contenu des tables

film						
<u>film_id</u>	title	release_year	length	rating	rental_rate	<u>category_id</u>
1	Academy Dinosaur	2006	86	PG	0.99	4
2	Bucket Brotherhood	2010	133	PG	4.99	1
3	Chocolate Duck	2014	92	G	2.99	2
4	Dancing Fever	2009	103	PG-13	2.99	2
5	Escape Metropolis	2018	141	R	4.99	1
6	Frost Head	2015	78	PG	0.99	5
7	Glacier Ocean	2021	118	G	3.99	3
8	Hunting Musketeers	2010	65	R	4.99	1
9	Island Empire	2019	NULL	PG-13	1.99	3
10	Jungle Closer	2006	128	R	0.99	5
11	Kissing Dolls	2013	96	G	2.99	4
12	Lonely Elephant	2022	112	PG	3.99	3

customer					
<u>customer_id</u>	<u>first_name</u>	<u>last_name</u>	<u>city</u>	<u>active</u>	<u>create_date</u>
1	Marie	Dupont	Aix-en-Provence	TRUE	2023-02-14
2	Lucas	Martin	Marseille	TRUE	2023-05-02
3	Sofia	Bernard	Aix-en-Provence	FALSE	2022-11-30
4	Hugo	Petit	Toulon	TRUE	2024-01-09
5	Léa	Moreau	Marseille	TRUE	2024-03-21
6	Adam	Girard	Aix-en-Provence	TRUE	2024-06-15
7	Nina	Roche	Nice	FALSE	2022-08-05
8	Paul	Faure	Toulon	TRUE	2025-01-12

actor		
actor_id	first	last
1	Nick	Wahlberg
2	Ed	Chase
3	Jennifer	Davis
4	Johnny	Lollobrigida
5	Bette	Nicholson
6	Grace	Mostel
7	Matthew	Johansson
8	Joe	Swank

category	
category_id	name
1	Action
2	Comédie
3	Documentaire
4	Animation
5	Horreur

film_actor	
film_id	actor_id
1	1
1	3
2	1
2	2
2	4
3	3
4	3
4	5
5	1
5	4
5	6
6	2
7	7
8	4
8	6
9	5
10	1
10	7
11	3

rental				
rental_id	rental_date	return_date	film_id	customer_id
1	2025-01-05	2025-01-08	2	1
2	2025-01-07	2025-01-12	5	1
3	2025-01-09	2025-01-11	6	2
4	2025-01-15	NULL	3	2
5	2025-01-18	2025-01-25	2	3
6	2025-02-02	2025-02-05	7	4
7	2025-02-04	NULL	10	4
8	2025-02-08	2025-02-09	1	5
9	2025-02-11	2025-02-18	4	5
10	2025-02-14	2025-02-16	5	5
11	2025-02-20	2025-02-27	6	6
12	2025-03-01	NULL	2	6
13	2025-03-03	2025-03-06	8	6
14	2025-03-07	2025-03-10	10	1
15	2025-03-12	2025-03-15	3	7
16	2025-03-15	NULL	9	2
17	2025-03-18	2025-03-20	5	4
18	2025-03-22	2025-03-29	1	6

payment				
payment_id	customer_id	rental_id	amount	payment_date
1	1	1	4.99	2025-01-08
2	1	2	4.99	2025-01-12
3	2	3	0.99	2025-01-11
4	3	5	4.99	2025-01-25
5	4	6	3.99	2025-02-05
6	5	8	0.99	2025-02-09
7	5	9	2.99	2025-02-18
8	5	10	4.99	2025-02-16
9	6	11	0.99	2025-02-27
10	6	13	4.99	2025-03-06
11	1	14	0.99	2025-03-10
12	7	15	2.99	2025-03-15
13	4	17	4.99	2025-03-20
14	6	18	0.99	2025-03-29

Une location non rendue n'a pas encore donné lieu à paiement.